

AI Assisted Coding

Lab Assignment – 11.1

Name : Likhith Sai

Hall.t.No. : 2303A51026

Batch : 01

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

demo1.py
1 """
2 Your college wants to develop a Campus Resource Management System
3 that handles:
4 1. Student Attendance Tracking Daily Log of students
5 entering/exiting the campus.
6 Implement code using hash table data structure to efficiently store and retrieve attendance records.
7 """
8 class HashTable:
9     def __init__(self, size=100):
10         self.size = size
11         self.table = [None] * self.size
12
13     def hash_function(self, key):
14         return hash(key) % self.size
15
16     def insert(self, key, value):
17         index = self.hash_function(key)
18         if self.table[index] is None:
19
PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
['Alice', 'Bob', 'Charlie']
['David', 'Eve']
['Alice', 'Bob', 'Charlie', 'David']
['Laptop', 'Smartphone']
['Smartphone']
['Smartphone']
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Attendance Tracking	Hash Table	It provides $O(1)$ average time complexity for lookups and updates. Using a Student ID as a key allows for near-instant logging of entry and exit times.
Event Registration	Binary Search Tree (BST)	A BST (or a balanced version like an AVL tree) keeps participant names sorted automatically. This allows for quick alphabetical searches and efficient removals if someone cancels.
Library Borrowing	Hash Table	Searching for a book by its ISBN or unique ID needs to be instantaneous. A Hash Table maps the ID directly to the book's status and due date without scanning the whole collection.
Bus Scheduling	Graph	Routes are essentially a series of nodes (stops) and edges (paths). A Graph allows the system to calculate the shortest path or find all stops connected to a specific route.
Cafeteria Queue	Queue	This follows the FIFO (First-In, First-Out) principle. The first student to line up should be the first one served, ensuring fairness and order in the dining hall.

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor with a file named `demo1.py`. The code defines a `Queue` class with methods `__init__`, `enqueue`, and `dequeue`. The `dequeue` method uses `pop(0)` to remove the first element, implementing a FIFO queue. Below the code, the terminal shows the execution of the script, which outputs "Patient A", "Patient B", and "None" when the queue is empty.

```

45 """
46 A hospital wants to develop a Smart Hospital Management System that handles:
47 1. Patient Check-In System Patients are registered and treated in order of arrival.
48 Implement code using a queue data structure to manage patient check-ins and ensure that patients are treated in the order they arrived.
49 """
50 class Queue:
51     def __init__(self):
52         self.items = []
53
54     def enqueue(self, item):
55         self.items.append(item)
56
57     def dequeue(self):
58         if not self.is_empty():
59             return self.items.pop(0)
60         return None # Queue is empty
61

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Patient A
Patient B
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Patient Check-In	Queue	Standard check-ins follow a FIFO (First-In, First-Out) rule. This ensures that non-emergency patients are seen in the exact order they arrived at the desk.
Emergency Handling	Priority Queue	In an ER, the "next" person isn't the one who arrived first, but the one with the highest medical urgency. A Priority Queue allows the system to always serve the most critical case first, regardless of arrival time.
Medical Records	Hash Table	When a doctor needs a patient's history, they need it instantly. Mapping a Patient ID to a record in a Hash Table provides $O(1)$ average time complexity for retrieval.
Appt. Scheduling	BST (or AVL Tree)	Appointments need to be kept in chronological order. A Binary Search Tree keeps them sorted by time, making it easy to find "the next available slot" or print a day's schedule in order.
Room Navigation	Graph	A hospital is a network of wards, floors, and corridors. A Graph represents these locations as nodes and the hallways as edges, allowing for "shortest path" navigation (e.g., from the ER to the ICU).

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor with a file named `demo1.py`. The code defines a `TrafficQueue` class with methods `enqueue` and `dequeue`. The `dequeue` method implements a FIFO (First-In, First-Out) principle. Below the code, a terminal window shows the execution of the script, which outputs "Vehicle 1", "Vehicle 2", and "None".

```

75 """
76 A city plans a Smart Traffic Management System that includes:
77 1. Traffic Signal Queue Vehicles waiting at signals
78 Implement code using a queue data structure to manage vehicles waiting at traffic signals, ensuring that vehicles are processed in
79 the order they arrived.
80 """
81 class TrafficQueue:
82     def __init__(self):
83         self.vehicles = []
84
85     def enqueue(self, vehicle):
86         self.vehicles.append(vehicle)
87
88     def dequeue(self):
89         if not self.is_empty():
90             return self.vehicles.pop(0)
91         return None # Queue is empty

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Vehicle 1
Vehicle 2
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Traffic Signal Queue	Queue	This follows the FIFO (First-In, First-Out) principle. The first vehicle to reach the red light should be the first to move when it turns green to maintain a fair and natural traffic flow.
Emergency Priority	Priority Queue	Ambulances and fire trucks cannot wait in a standard line. A Priority Queue ensures these vehicles are bumped to the front of the "processing" list regardless of when they arrived at the intersection.
Vehicle Lookup	Hash Table	Police or automated cameras need to link a license plate to a driver instantly. A Hash Table provides $O(1)$ average time complexity, allowing for near-instant retrieval among millions of records.
Road Network	Graph	A city is a collection of intersections (nodes) and roads (edges). A Graph is the only way to logically map these connections and run algorithms like Dijkstra's to find the fastest route across town.
Parking Slots	Hash Table	To track whether slot "A-101" is occupied, a Hash Table (or a simple Bit Map/Array if IDs are contiguous) allows for instant

Feature	Data Structure	Justification
		status checks and updates without searching through the whole lot.

Task Description #12: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor with a file named `demo1.py`. The code defines a `Node` class and a `ShoppingCart` class. The `Node` class has an `__init__` method that takes a `product` and sets `self.product` and `self.next` to `None`. The `ShoppingCart` class has an `__init__` method that sets `self.head` to `None` and an `add_product` method that creates a new `Node` and adds it to the `head` of the list. The terminal window shows the command `python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"` being executed, and the output shows the list `['Laptop', 'Smartphone']` being printed.

```

105 """
106 An e-commerce company wants to build a Smart Online Shopping System with:
107 1. Shopping Cart Management Add and remove products dynamically.
108 Implement code using a linked list data structure to manage the shopping cart, allowing customers to add and remove products
    efficiently.
109 """
110 class Node:
111     def __init__(self, product):
112         self.product = product
113         self.next = None
114 class ShoppingCart:
115     def __init__(self):
116         self.head = None
117
118     def add_product(self, product):
119         new_node = Node(product)
120         if self.head is None:
121             self.head = new_node

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
['Laptop', 'Smartphone']
['Smartphone']
['Smartphone']
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Shopping Cart	Linked List	A Doubly Linked List allows for $O(1)$ additions and removals of products at any point. This is ideal for a dynamic cart where users might remove an item from the middle of their list without re-indexing the entire collection.
Order Processing	Queue	Orders must be handled using the FIFO (First-In, First-Out) principle to ensure fairness. The first customer to click "Checkout" is the first whose order is sent to the warehouse for packing.
Top-Selling Tracker	Priority Queue	A Max-Priority Queue (often implemented as a Heap) keeps the products with the highest sales counts at the top. This allows the platform to instantly retrieve the "Top N" trending products for the homepage.
Search Engine	Hash Table	When a user enters a specific Product ID or SKU, the system needs to find that product instantly among millions. A Hash Table provides $O(1)$ average time complexity for these direct lookups.
Delivery Routes	Graph	Warehouses and delivery addresses are nodes, while roads are edges. A Graph structure is necessary to calculate the most

Feature	Data Structure	Justification
		efficient path for drivers, often using algorithms like Dijkstra's or A*.

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

161
162
163 """
164 A logistics company wants to implement:
165 1. Shipment Processing
166 Implement code using a stack data structure to manage shipment processing, allowing the company to process shipments in a last-in-first
167 """
168 class Stack:
169     def __init__(self):
170         self.shipments = []
171
172     def push(self, shipment):
173         self.shipments.append(shipment)
174
175     def pop(self):
176         if not self.is_empty():
177             return self.shipments.pop()
178         return None # Stack is empty
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Feature	Data Structure	Justification
Shipment Processing	Queue	Standard shipments are processed using the FIFO (First-In, First-Out) principle. This ensures that the first package to arrive at the sorting facility is the first one loaded onto a truck.
Urgent Shipment Priority	Priority Queue	"Express" or "Overnight" packages must jump the line. A Priority Queue allows the system to always extract the highest-priority item next, regardless of when it entered the system.
Product Lookup by Code	Hash Table	Warehouses contain millions of SKUs. A Hash Table allows a worker to scan a barcode and retrieve the product's aisle, shelf, and quantity in $O(1)$ average time.
Warehouse Network	Graph	Logistics hubs and distribution centers are nodes connected by transport routes (edges). A Graph is essential for calculating the most cost-effective path between two geographical points.
Inventory Operations	Linked List	For a dynamic inventory where products are frequently inserted or removed from various points in a list (not just the ends), a Linked List offers flexible $O(1)$ pointer updates without shifting elements.

Task Description #14: Smart Banking Management System – Data Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor with a file named `demo1.py`. The code defines a `CustomerQueue` class with methods for initializing a queue, enqueueing customers, and dequeueing customers. The terminal output shows the execution of the script, which prints 'Customer 1', 'Customer 2', and 'None'.

```

191
192 """
193 A bank plans to develop a Smart Banking System that includes:
194 1. Customer Service Token System Customers are served in order of arrival.
195 Implement code using a queue data structure to manage customer service tokens, ensuring that customers are served in the order they arrive.
196 """
197 class CustomerQueue:
198     def __init__(self):
199         self.customers = []
200
201     def enqueue(self, customer):
202         self.customers.append(customer)
203
204     def dequeue(self):
205         if not self.is_empty():
206             return self.customers.pop(0)
207         return None # Queue is empty
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
277
```

Feature	Data Structure	Justification
		cash replenishment routes or analyze the connectivity between different city branches.

Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.
- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

The screenshot shows a VS Code editor with a file named `demo1.py`. The code defines a `HashTable` class with methods for initialization, hashing, and insertion. The terminal window shows the command `python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"` being executed, with the output showing the insertion of a password into the hash table.

```

221 """
222 An online education platform wants to develop a Learning Management System that handles:
223 1. Student Login Authentication [ ] Quick validation of user credentials.
224 Implement code using a hash table data structure to efficiently store and retrieve student login credentials for authentication purposes
225 """
226 class HashTable:
227     def __init__(self, size=100):
228         self.size = size
229         self.table = [None] * self.size
230
231     def hash_function(self, key):
232         return hash(key) % self.size
233
234     def insert(self, key, value):
235         index = self.hash_function(key)
236         if self.table[index] is None:

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
password123
securepass
newpassword
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Login Authentication	Hash Table	Student credentials (usernames/hashed passwords) need to be validated instantly. A Hash Table provides $O(1)$ average time complexity, ensuring that login speed remains constant even as the student body grows.
Assignment Submission	Queue	This follows the FIFO (First-In, First-Out) principle. It ensures that the grading system or the storage processor handles assignments in the exact order they were uploaded by students.
Top Performer Ranking	Priority Queue	Ranking students by scores is most efficient with a Max-Heap. A Priority Queue allows the platform to instantly extract the highest-scoring students for a leaderboard without sorting the entire database.
Course Prerequisites	Graph	Courses and their dependencies form a complex network. A Directed Acyclic Graph (DAG) is perfect for representing that Course A must be completed before Course B, allowing for topological sorting.
Undo Feature (Editor)	Stack	This relies on the LIFO (Last-In, First-Out) principle. Every action is "pushed" onto the stack; when a student hits

Feature	Data Structure	Justification
		"Undo," the most recent action is "popped" off to revert the state.

Task Description #16: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

The screenshot shows a VS Code workspace with a file named `demo1.py`. The code defines a `PassengerQueue` class with methods for enqueueing and dequeueing passengers. The terminal output shows the execution of the script, which prints "Passenger A", "Passenger B", and "None".

```

265 """
266 An airport authority plans to implement:
267 1. Passenger Check-In Queue: Process passengers in order of arrival.
268 Implement code using a queue data structure to manage passenger check-ins, ensuring that passengers are processed in the order they
   arrived.
269 """
270 class PassengerQueue:
271     def __init__(self):
272         self.passengers = []
273
274     def enqueue(self, passenger):
275         self.passengers.append(passenger)
276
277     def dequeue(self):
278         if not self.is_empty():
279             return self.passengers.pop(0)
280         return None # Queue is empty
281
282 """
283 """

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Passenger A
Passenger B
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Passenger Check-In	Queue	Check-in counters operate on a FIFO (First-In, First-Out) basis. This ensures that the first passenger to join the line is the first one assisted, maintaining order and fairness during peak travel times.
Emergency Landing	Priority Queue	In aviation, safety overrides arrival order. A Priority Queue allows the Air Traffic Control system to automatically move aircraft with emergency codes (e.g., low fuel, engine failure) to the top of the landing list.
Flight Lookup	Hash Table	Travelers and staff need to find flight status instantly among hundreds of daily departures. A Hash Table using the Flight ID as a key provides $O(1)$ average time complexity for these searches.
Air Route Connectivity	Graph	Global aviation is a giant network. Airports are nodes and flight paths are edges; a Graph is necessary to calculate connecting flights, total travel distance, and fuel requirements.
Boarding (Luggage)	Stack	Overhead bins and narrow cabin aisles often function on a LIFO (Last-In, First-Out) basis. The last bag pushed into a

Feature	Data Structure	Justification
		deep storage compartment or the last person in a narrow jet bridge is often the first one to come out.

Task Description #17: Smart Social Media Platform – Data Structure

Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments

```

294
295 """
296 A social media company wants to develop a system that includes:
297 1. News Feed Generation [ ] Display posts chronologically.
298 Implement code using a linked list data structure to manage the news feed, allowing for efficient insertion of new posts and
299 retrieval of posts in chronological order.
300 """
301 class PostNode:
302     def __init__(self, post):
303         self.post = post
304         self.next = None
305 class NewsFeed:
306     def __init__(self):
307         self.head = None
308
309     def add_post(self, post):
310         new_node = PostNode(post)
311         if self.head is None:
312             self.head = new_node
313         else:
314             current = self.head
315             while current.next:
316                 current = current.next
317             current.next = new_node
318
319     def get_news_feed(self):
320         posts = []
321         current = self.head
322         while current:

```

Feature	Data Structure	Justification
News Feed	Linked List	A Doubly Linked List is excellent for chronologically ordered data. It allows for $O(1)$ insertions at the head (newest post) and efficient traversal to load "older" posts as the user scrolls down.
Trending Hashtags	Priority Queue	To find what's "trending," the system needs to constantly track the most frequently used hashtags. A Max-Heap (Priority Queue) allows the top N hashtags to be extracted instantly without sorting the entire dataset.
Profile Search	Hash Table	Username are unique identifiers. Mapping a username to a profile object in a Hash Table ensures that searching for a friend happens in $O(1)$ average time, regardless of how many millions of users exist.
Friend Network	Graph	Social connections are non-linear and multidimensional. A Graph (using an Adjacency List) perfectly represents users as nodes and friendships as edges, enabling features like "Suggested Friends" or "Degrees of Separation."
Message Undo	Stack	This follows the LIFO (Last-In, First-Out) principle. By pushing sent messages onto a Stack, the system can easily "pop" the most

Feature	Data Structure	Justification
		recent message to remove it if the user hits the undo/withdraw button.

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings

```

331
332
333
334 """
335 A university wants to build a hostel management system that manages:
336 1. Room Allotment
337 Implement code using a hash table data structure to efficiently store and retrieve room allotment information for students.
338 """
339 class HashTable:
340     def __init__(self, size=100):
341         self.size = size
342         self.table = [None] * self.size
343
344     def hash_function(self, key):
345         return hash(key) % self.size
346
347     def insert(self, key, value):
348         index = self.hash_function(key)
349         if self.table[index] is None:
350             self.table[index] = [(key, value)]
351         else:
352             for i, (k, v) in enumerate(self.table[index]):
353                 if k == key:
354                     self.table[index][i] = (key, value) # Update existing key
355             return
356             self.table[index].append((key, value)) # Add new key
357
358     def retrieve(self, key):
359         index = self.hash_function(key)

```

Feature	Data Structure	Justification
Room Allotment	Queue	Rooms are generally allotted on a "First-Come, First-Served" basis during the registration period. A Queue ensures that the student who applied first gets the first choice of available rooms.
Medical Emergency	Priority Queue	In a hostel, health crises require immediate attention. A Priority Queue allows the system to process medical alerts based on severity (e.g., an unconscious student vs. a minor fever) rather than arrival time.
Student Record Retrieval	Hash Table	Retrieving a student's profile, contact info, or fee status by their Roll Number must be near-instant. A Hash Table provides $O(1)$ average time complexity, making lookups efficient regardless of hostel size.
Hostel Block Connectivity	Graph	A large campus has multiple blocks, messes, and security gates connected by pathways. A Graph represents these locations as nodes and paths as edges, helping in planning security patrols or maintenance routes.
Visitor Entry Log	Linked List	Visitors are logged sequentially as they enter and leave. A Linked List (or a Doubly Linked List) allows for efficient

Feature	Data Structure	Justification
		appending of new entries and is flexible for time-based tracking without requiring a fixed memory block.

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor window with a file named `demo1.py`. The code defines a `ComplaintQueue` class with methods for initializing a queue, enqueueing complaints, and dequeueing complaints. The terminal output shows the execution of the script, which prints the details of two complaints and returns `None` after the second dequeue operation.

```
375 """
376 An electricity board plans to develop a system that includes:
377 1. Complaint Handling
378 Implement code using a queue data structure to manage customer complaints, ensuring that complaints are addressed in the order they
379 were received.
380 """
381 class ComplaintQueue:
382     def __init__(self):
383         self.complaints = []
384     def enqueue(self, complaint):
385         self.complaints.append(complaint)
386     def dequeue(self):
387         if not self.is_empty():
388             return self.complaints.pop(0)
389         return None # Queue is empty
390
```

Terminal Output:

```
PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Complaint 1: Power outage in area A
Complaint 2: Billing issue for customer B
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>
```

Feature	Data Structure	Justification
Complaint Handling	Queue	General complaints (like billing queries) should be addressed in the order they were received. A Queue follows the FIFO (First-In, First-Out) principle, ensuring fair and organized customer service.
Fault Priority Handling	Priority Queue	Not all power outages are equal; a total blackout at a hospital takes precedence over a flickering streetlamp. A Priority Queue allows the system to dispatch repair crews to critical faults first.
Consumer Record Lookup	Hash Table	With millions of consumers, finding details by a Consumer ID must be instantaneous. A Hash Table provides $O(1)$ average time complexity for retrieval, making it the fastest choice for direct lookups.
Power Grid Connectivity	Graph	The grid consists of power plants, substations, and transformers (nodes) connected by transmission lines (edges). A Graph is necessary to model flow, identify points of failure, and reroute power during maintenance.
Meter Reading History	Linked List	Meter readings are sequential data points recorded over time. A Linked List (or a Doubly Linked List) allows for efficient

Feature	Data Structure	Justification
		appending of new monthly readings and chronological traversal of usage history.